



Manual do Dev: Inserindo o Bloco de Giro (PyCozmo)

O Fluxo da Informação

A informação segue esta hierarquia:

1. **Visual** (Bloco no HTML) → 2. **Tradução** (JavaScript no Renderer) → 3. **Ponte** (Serviço Node.js) → 4. **Ação** (Script Python).

Passo 1: A Interface (Onde o bloco vive)

Arquivo: `renderer/views/home/home.html`

Adicione o bloco no menu lateral para que ele apareça para o usuário.

XML

```
<category name="Movimentos" colour="#5b67a5">
  <block type="cozmo_forward"></block>
  <block type="cozmo_turn">
    <field name="ANGLE">90</field>
  </block>
  <block type="cozmo_stop"></block>
</category>
```

Passo 2: A Definição do Bloco (O que ele gera)

Arquivo: `renderer/blocks_device/cozmo_blocks/cozmo_motions.js`

Aqui você desenha o bloco e define o texto que ele vai "escrever". **Cuidado:** Não declare a constante `cozmoGenerator` mais de uma vez!

JavaScript

```
// 1. O Desenho do bloco
Blockly.Blocks['cozmo_turn'] = {
  init: function() {
    this.appendDummyInput()
      .appendField("Cozmo: Girar")
      .appendField(new Blockly.FieldNumber(90), "ANGLE")
      .appendField("graus");
```

```

    this.setPreviousStatement(true, null);
    this.setNextStatement(true, null);
    this.setColour(230);
  }
};

// 2. A Tradução (O que vira código)
cozmoGenerator.forBlock['cozmo_turn'] = function(block) {
  const angle = block.getFieldValue('ANGLE');
  // O nome 'virar' deve ser idêntico ao que está no blockly-service.js
  return `await virar(${angle});\n`;
};

```

Passo 3: O Serviço de Execução (A Ponte)

Arquivo: `main/services/blockly-service.js`

A VM (Máquina Virtual) lê o texto `await virar(90)` e precisa saber o que fazer.

JavaScript

```

const cozmoActions = {
  // ... outras ações ...
  virar: async (angulo) => {
    // Criamos o pacote de dados (JSON)
    const comando = { cmd: "turn", angle: parseInt(angulo) };

    // Enviamos para o processo principal que fala com o Python
    enviarParaPython(comando);

    // IMPORTANTE: Damos tempo para o robô físico terminar o movimento
    // Ex: 90 graus leva cerca de 1.5 segundos
    return new Promise(resolve => setTimeout(resolve, 1500));
  }
};

```

Passo 4: O Controle do Robô (A Realidade)

Arquivo: `python/cozmo_server.py`

O Python recebe o JSON e comanda os motores. Como o `Client` do PyCozmo não tem `turn_in_place` direto, usamos as rodas em direções opostas.

Python

```

elif cmd == "turn":

```

```
angle = data.get("angle", 0)
turn_speed = 30 # Velocidade constante para o giro

# Cálculo: quanto tempo as rodas devem girar?
# Dividir o ângulo por 85 é uma estimativa para 30 de velocidade
duration = abs(angle) / 85.0

if angle > 0:
    # Ângulo positivo: roda esquerda recua, direita avança (Gira Esquerda)
    cli.drive_wheels(-turn_speed, turn_speed, duration=duration)
else:
    # Ângulo negativo: roda esquerda avança, direita recua (Gira Direita)
    cli.drive_wheels(turn_speed, -turn_speed, duration=duration)

log({"status": "ok", "cmd": "turn"})
```

Checklist de Erros Comuns para o Júnior:

1. **Identifier already declared:** Você copiou `const cozmoGenerator` de novo. Apague a linha repetida.
2. **Unknown Command:** Você mudou o nome no JS (`await virar`) mas esqueceu de mudar no Python (`if cmd == "turn"`). Os nomes devem bater.
3. **O robô não para:** Lembre-se que no PyCozmo, se você não definir `duration` ou não chamar `stop_all_motors()`, ele continuará girando para sempre.